

Introduction to Transformers

Procheta Sen

April 19, 2026

What is a Transformer?

- ▶ A deep learning architecture based on *Attention*, introduced in the paper titled 'Attention is All You Need' in NeurIPS 2017.
- ▶ Core of today's NLP models: BERT, GPT, T5, translation.
- ▶ **Background:**

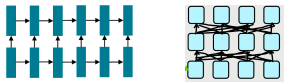


Figure: Sequence to Sequence Processing RNN/LSTM Vs. Transformers

- ▶ RNNs struggled with long Sequence.
- ▶ LSTMs couldn't process tokens in Parallel
- ▶ **Advantages:**
 - ▶ Captures long-range dependencies effectively and trains efficiently in parallel
 - ▶ Achieves state-of-the-art results across NLP tasks.

How Transformer works?

1. Transformer has primarily two components: Encoder, Decoder. Encoder has self attention and decoder has masked self attention.
2. The encoder builds rich, contextual representations of the input sequence
3. *Decoder* The output of decoder is a set of probabilities on a vocabulary set, generates tokens step by step.
4. **Variations in Architecture:** Encoder-only (e.g., BERT) $\Rightarrow$ for understanding tasks , Decoder-only (e.g., GPT) $\Rightarrow$ for text generation, Encoder–Decoder (e.g., T5, original Transformer) $\Rightarrow$ for seq2seq tasks.

Transformer Block Diagram

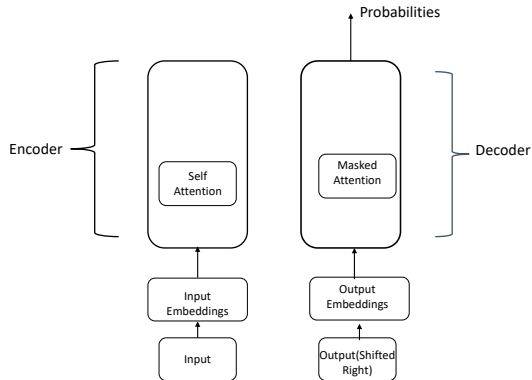


Figure: Transformer Block Diagram

Transformer Architecture Breakdown

- ▶ Both encoder and decoder comprises 6 identical layers as shown in Figure.
- ▶ Three important components: Residual Connection, Layer normalisation and positional encoding .
- ▶ *Positional Encoding* – adds word order information.
- ▶ Directly passing 'raw' embeddings to the next layer can help preventing forgetting/distortion of information to the next layer, residual connections were proposed. $x_{l+1} = F(x_{l-1}) + x_l$
- ▶ layer norm essentially normalizes the output of feed forward layer
- ▶ *Residual Connections + Layer Norm* – stabilize training.

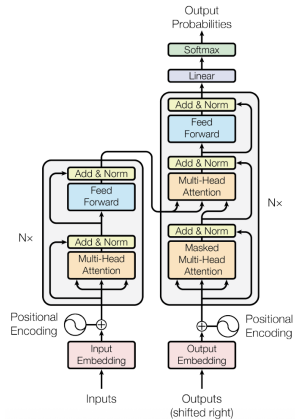


Figure: Vashwani et al. 2017

Deep Dive to Attention Mechanisms: Part 1

- ▶ Attention is Like highlighting important words while reading a paragraph..
Example: In “The cat sat on the mat”, to understand “cat”, the words “sat” and “mat” are more relevant than “the”.
- ▶ **Mechanism (high-level):** Each word builds a query, key, and value. Similarity between query and keys decides which values to “attend to.”
- ▶ **Intuition:** Query: ‘what this position is looking for.’
Key: ‘what each token offers to be matched on.’
Value: Weighted average of the query-key similarity values.

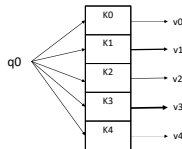


Figure: Overview of Attention

Deep Dive to Attention Mechanisms: Part 2

Scaled Dot-Product Attention Parallelizes the operation. Attention is computed on n number of queries simultaneously

Mathematical Representation:

$$\mathbf{Q} = \mathbf{XW}_Q, \quad \mathbf{K} = \mathbf{XW}_K,$$

$$\mathbf{V} = \mathbf{XW}_V$$

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right)$$

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{AV}$$

- ▶ **Scaling** $1/\sqrt{d_k}$ keeps dot-products numerically stable.
- ▶ **Multi-head**: compute multiple (Q, K, V) in parallel and concatenate.

Example Illustration

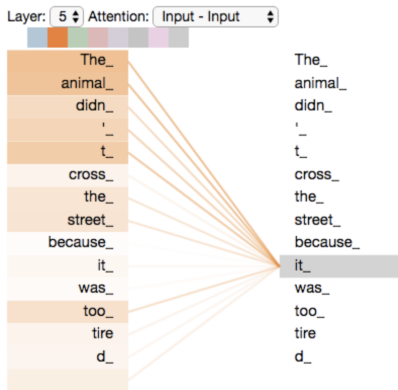


Figure: Attention pattern for the word 'it' in the sentence 'The animal didn't cross the street because it was too tired' (Jay Alammar Transformer Illustrations)

Pretrain / Finetuning

▶ **Pretraining:**

- ▶ Train a neural network to perform language modeling on a large amount of text.
- ▶ Save the network parameters.
- ▶ Captures general linguistic patterns: syntax, semantics, discourse.

▶ **Fine-tuning:**

- ▶ Adapt the pretrained model to a specific downstream task (e.g., sentiment analysis, translation, QA).
- ▶ Requires relatively small labeled datasets.
- ▶ Often only a few epochs are sufficient.

▶ **Benefits:**

- ▶ Dramatic reduction in task-specific data requirements.
- ▶ Faster convergence and higher accuracy.
- ▶ Enables transfer learning across domains and languages.

Important Questions and Bottlenecks

- ▶ **Scalability:** How can we efficiently train ever-larger Transformers (billions of parameters) without prohibitive compute and energy costs?
- ▶ **Data requirements:** Do we always need massive text corpora? How do we adapt models to low-resource languages and domains?
- ▶ **Interpretability:** What exactly do attention weights capture? Can we reliably explain model predictions?
- ▶ **Deployment bottlenecks:** Memory footprint and latency in real-world applications (e.g., mobile devices, chatbots).